

Open `libcflang.so` with Ghidra. Decompile

`Java_de_tadris_flang_ui_fragment_NativeSettingsBridge_resolveSecret`.

It opens `cover_preview.png`.

```
11 __s1 := (Char"/0x0");
if (((param_4 != 0) && (lVar11 = AAssetManager_fromJava(param_1,param_4), lVar11 != 0)) &&
    (lVar11 = AAssetManager_open(lVar11,"opening_book/cover_preview.png" 3), lVar11 != 0)) {
    uVar6 = AAsset_getLength();
    if (0 < (long)uVar6) {
```

Here is `CTF0`.

```
if (0 < iVar5) {
    local_98 = CONCAT35(local_98._5_3_0x3046544308);
    pcVar19 = (char *)((ulong)&local_80 | 1);
```

Input
08 43 54 46 30
ABC 14 1
Output
BS CTF0

Here is to find `END0`.

```
LAB_00151178:
    piVar8 = memchr(__s,0x45,lVar11 - 3);
    if (piVar8 == (int *)0x0) goto LAB_001511f8;
    if (*piVar8 != 0x30444e45) goto code_r0x00151198;
    uVar16 = (long)piVar8 - (long)pcVar19;
    if (iVar5 < uVar16) {
```

Input
45 4E 44 30
ABC 11  1
Output
END0

I guess these are to calculate the offset of `CTF0` and `END0`? They all minus `pcVar19` so it may be like a base address. Therefore I guess flag is between `CTF0` and `END0` from the literal meaning.

```
while (piVar8 = memchr(pcVar7,0x43,uVar12 - 3), piVar8 != (int *)0x0) {
    if (*piVar8 == local_98._1_4_) {
        if ((piVar8 != piVar1) && ((long)piVar8 - (long)pcVar19 != -1)) {
            uVar12 = ((long)piVar8 - (long)pcVar19) + 4;
            if (uVar12 <= uVar6) {
                pcVar7 = pcVar19 + uVar12;
                lVar11 = (long)piVar1 - (long)pcVar7;
                __s = pcVar7;
            }
        }
        LAB_00151178:
        piVar8 = memchr(__s,0x45,lVar11 - 3);
        if (piVar8 == (int *)0x0) goto LAB_001511f8;
        if (*piVar8 != 0x30444e45) goto code r0x00151198;
        uVar16 = (long)piVar8 - (long)pcVar19;
        if (piVar8 == piVar1) {
```

This part is just to xor 0x33 byte by byte but optimized by the compiler. If the length is less than 8 then xor 0x33 byte by byte directly, else xor 0x33 8 bytes per group.

```

if (uVar6 == 0) goto LAB_001511fc;
pbVar14 = pbVar18;
if (uVar6 < 8) goto LAB_0015132c;
if (uVar6 < 0x20) {
    uVar12 = 0;
    goto LAB_001512f4;
}
uVar12 = uVar6 & 0xffffffffffffffe0;
puVar13 = (ulong *) (pbVar18 + 0x10);
uVar16 = uVar12;
do {
    uVar16 = uVar16 - 0x20;
    puVar13[-1] = puVar13[-1] ^ 0x3333333333333333;
    puVar13[-2] = puVar13[-2] ^ 0x3333333333333333;
    puVar13[1] = puVar13[1] ^ 0x3333333333333333;
    *puVar13 = *puVar13 ^ 0x3333333333333333;
    puVar13 = puVar13 + 4;
} while (uVar16 != 0);
if (uVar6 != uVar12) {
    if ((uVar6 & 0x18) == 0) {
        pbVar14 = pbVar18 + uVar12;
    }
    else {
LAB_0015132c:
        do {
            pbVar15 = pbVar14 + 1;
            *pbVar14 = *pbVar14 ^ 0x33;
            pbVar14 = pbVar15;
        } while (pbVar15 != pbVar18 + uVar6);
    }
}

```

So now we know: the information is in `cover_preview.png`, the flag is between `CTF0` and `END0`, and it xor `0x33` byte by byte.

I wrote a script for this:

```

1 data = open("cover_preview.png", "rb").read()

```

```

2
3 start = data.index(b"CTF0")
4 end = data.index(b"END0")
5
6 str = []
7 for i in range(start, end):
8     str.append(data[i] ^ 0x33)
9
10 print(str)
11
12 flag = ""
13 for b in str:
14     flag += chr(b)
15
16 print(flag)

```

```

(mahiro@kali) - [~/Homework/defense/cover_preview]
$ python3 exploit.py
[112, 103, 117, 3, 110, 56, 105, 118, 101, 95, 112, 97, 115, 115, 124, 70, 76, 65, 71, 123, 110, 56, 105, 118, 101, 95, 102, 108, 97, 103, 125]
pgun8ive_pass{FLAG{n8ive_flag}

```